



Smart Refrigerator Monitoring

EXERCISE NO – 02

HARRISH T M	– 24MER016
KAPIL P	– 24MER022
RAMPRASATH D	– 24MER047
SARAVANAKARTHIK S R	– 24MER055
SETHUPATHI M	– 24MER056

2. Smart Refrigerator Monitoring

1. Project overview:

The Smart Refrigerator Monitoring System is designed to continuously monitor the internal temperature and humidity of a refrigerator. Maintaining suitable temperature and humidity conditions is important for preserving food quality, reducing spoilage, and improving the overall efficiency of refrigerator operation.

The proposed system uses a DHT11 temperature and humidity sensor to measure the environmental conditions inside the refrigerator. An ESP32 microcontroller is used to collect and process the sensor readings. The ESP32 can transmit the collected information through Wi-Fi to an IoT platform, allowing the user to monitor the refrigerator remotely.

An OLED display can also be used to show the current temperature and humidity locally. If the temperature rises above the predefined limit, the system can activate an LED or buzzer to warn the user.

The system can continuously record the environmental conditions, allowing the user to identify unusual temperature changes. This information can be useful for detecting problems such as frequent door opening, poor cooling, excessive loading, or possible refrigerator malfunction.

The proposed system provides a simple and economical approach to smart refrigerator monitoring. It can also be expanded in the future by adding energy monitoring, automatic cooling control, mobile notifications, cloud data storage, and predictive maintenance.

2. Problem Statement:

Food stored inside a refrigerator requires suitable temperature conditions to maintain freshness and prevent premature spoilage. In a conventional refrigerator, the user may not continuously know the actual internal temperature and humidity conditions.

Temperature can increase because of frequent door opening, improper loading, power interruptions, insufficient cooling, or faults in the refrigeration system. If the abnormal condition is not noticed quickly, perishable food may lose its quality.

Another problem is unnecessary energy consumption. If the refrigerator operates continuously due to unsuitable conditions, it may consume more electrical energy. Manual checking of refrigerator conditions also does not provide continuous historical information.

Therefore, there is a need for a smart monitoring system that can continuously measure refrigerator temperature and humidity and provide immediate information to the user.

The proposed system solves this problem by using a DHT11 sensor and ESP32 microcontroller. The sensor measures the internal environmental conditions, while the ESP32 processes and transmits the data. The system can generate an alert whenever the temperature or humidity exceeds the selected limits.

3. Proposed Solution:

The proposed system consists of a DHT11 sensor, ESP32 microcontroller, OLED display, LED, buzzer and IoT platform.

The DHT11 sensor is installed at a suitable location inside the refrigerator. It continuously measures the internal temperature and relative humidity. The sensor readings are transferred to the ESP32.

The ESP32 processes the received values and displays them on an OLED screen. Through the built-in Wi-Fi capability of the ESP32, the readings can also be transmitted to an IoT cloud platform.

The user can view the refrigerator conditions through a web or mobile dashboard. If the temperature rises above the selected threshold, the system generates a warning.

For example:

- Normal temperature → System operates normally.
- High temperature → Warning is generated.
- High humidity → Humidity warning is generated.
- Sensor failure → System indicates an error.

The proposed solution therefore provides continuous monitoring, remote access, warning indication and historical data analysis.

4. System Architecture:

The system architecture explains the connection and communication between the different components.

The DHT11 sensor acts as the sensing unit. It measures the temperature and humidity inside the refrigerator. The sensor sends the measured values to the ESP32 microcontroller.

The ESP32 processes the sensor readings and displays them on the OLED display. It also uses its built-in Wi-Fi module to transmit the data to the IoT cloud platform.

The cloud platform stores and processes the collected information. The user can access the data through a mobile phone or computer using an IoT dashboard.

The complete system flow is:

DHT11 Sensor → ESP32 → Wi-Fi → IoT Cloud → Web/Mobile Dashboard → User

An additional alert system is connected to the ESP32. When the temperature or humidity crosses the predefined limit, the ESP32 activates the LED and buzzer.

Main functions of the architecture

- Temperature measurement
- Humidity measurement
- Data processing

- Local display
- Wireless communication
- Cloud monitoring
- Alert generation
- Data storage
- Performance analysis

5.Circuit Table

Component	Component Pin	ESP32 Pin
DHT22	VCC	3.3V
DHT22	GND	GND
DHT22	DATA	GPIO 4
OLED	VCC	3.3V
OLED	GND	GND
OLED	SDA	GPIO 21
OLED	SCL	GPIO 22
LED	Anode (+)	GPIO 2
LED	Cathode (-)	GND
Buzzer	Positive (+)	GPIO 5
Buzzer	Negative (-)	GND

6. Circuit Design:

The circuit is designed around the ESP32 development board. The ESP32 is selected because it has built-in Wi-Fi and sufficient input/output pins for connecting the required components.

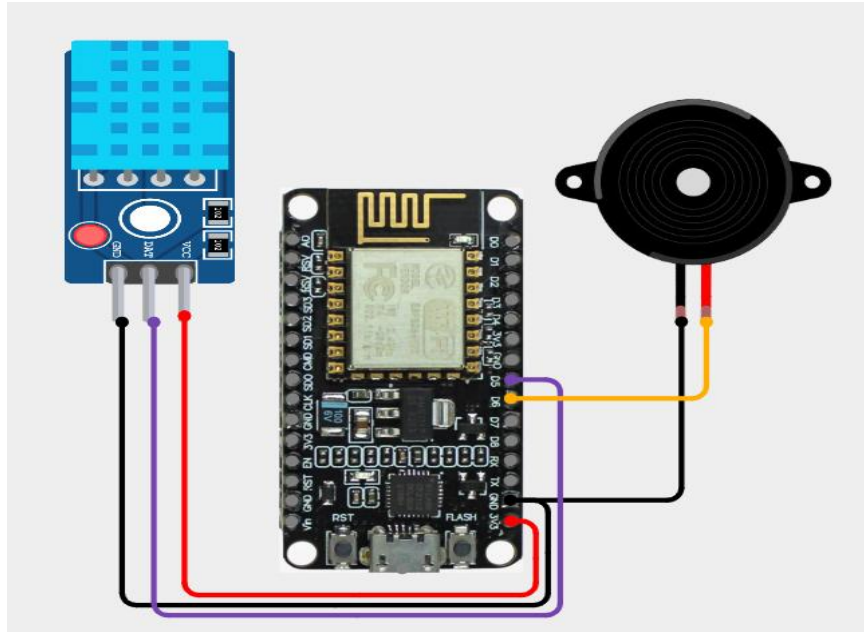
The DHT11 sensor is connected to GPIO 4. Its VCC pin is connected to the 3.3 V supply and the GND pin is connected to the ESP32 ground.

The OLED display is connected using the I²C communication interface. The SDA pin is connected to GPIO 21 and the SCL pin is connected to GPIO 22.

An LED is connected to GPIO 2 for visual indication. A buzzer is connected to GPIO 5 for audible warning.

During normal operation, the LED and buzzer remain OFF. When the temperature or humidity exceeds the selected limit, the ESP32 activates the warning devices.

The circuit operates using low-voltage DC power. The electronic components should be properly protected from moisture and condensation inside the refrigerator.



7. Algorithm:

1. Power ON the system:

Switch ON the refrigerator monitoring system.

All connected components receive power and start operating.

2. Initialize ESP32 and DHT11:

ESP32 starts the control program and initializes the DHT11 sensor.

The sensor becomes ready to measure temperature and humidity.

3. Establish Wi-Fi connection:

ESP32 connects to the available Wi-Fi network.

This allows the system to send data to the IoT dashboard.

4. Measure refrigerator temperature:

DHT11 senses the temperature inside the refrigerator.

The measured value is received by the ESP32.

5. Measure humidity level:

DHT11 measures the humidity inside the refrigerator.

The humidity reading is sent to the ESP32 for processing.

6. Display values on OLED:

ESP32 processes the temperature and humidity readings.

The values are displayed on the OLED screen for easy monitoring.

7. Compare with preset limits:

The measured values are compared with predefined safe limits.

This helps identify abnormal temperature or humidity conditions.

8. Activate the buzzer:

If the measured values exceed the preset limits, the buzzer is activated.
This provides an alert to the user about the abnormal condition.

9. Send data to IoT dashboard:

ESP32 sends the sensor readings through Wi-Fi to the IoT dashboard.
The user can monitor refrigerator conditions remotely.

10. Record and repeat readings:

The readings are continuously recorded for monitoring and analysis.
The process repeats automatically to provide real-time monitoring.

8. Coding:

```
#include <ESP8266WiFi.h>
#include "AdafruitIO_WiFi.h"
#include <DHT.h>
#define WIFI_SSID "K-APIL"
#define WIFI_PASS "kapil.2333"#define IO_USERNAME "saravanakarthiks"
#define IO_KEY "aio_idPg82y91fmNVXL0iJA71zKwH97q"
AdafruitIO_WiFi io(IO_USERNAME, IO_KEY, WIFI_SSID, WIFI_PASS);
AdafruitIO_Feed *temperatureFeed =
  io.feed("temperature");
AdafruitIO_Feed *humidityFeed =
  io.feed("humidity");
AdafruitIO_Feed *refrigeratorStatusFeed =
  io.feed("refrigerator-status");
AdafruitIO_Feed *buzzerStatusFeed =
  io.feed("buzzer-status");
#define DHT_PIN 14
#define BUZZER_PIN 12
#define DHT_TYPE DHT11
DHT dht(DHT_PIN, DHT_TYPE);

float TEMP_MIN = 2.0;
float TEMP_MAX = 8.0;
float HUM_MIN = 30.0;
float HUM_MAX = 70.0;
#define BUZZER_ON HIGH
#define BUZZER_OFF LOW
void setup()
{
  Serial.begin(115200);
  pinMode(BUZZER_PIN, OUTPUT);
  digitalWrite(BUZZER_PIN, BUZZER_OFF);
```

```

dht.begin();
Serial.println();
Serial.println();
Serial.println();
Serial.println();
Serial.println("Connecting to Adafruit IO...");
io.connect();
while (io.status() < AIO_CONNECTED)
{
    Serial.print(".");
    delay(500);
}
Serial.println();
Serial.println("Adafruit IO Connected!");
Serial.println();
Serial.print("Temperature Limit: ");
Serial.print(TEMP_MIN);
Serial.print(" - ");
Serial.print(TEMP_MAX);
Serial.println(" °C");
Serial.print("Humidity Limit: ");
Serial.print(HUM_MIN);
Serial.print(" - ");
Serial.print(HUM_MAX);
Serial.println(" %");
Serial.println();
void loop()
{
    io.run();
    float temperature = dht.readTemperature();
    float humidity = dht.readHumidity();
    if (isnan(temperature) || isnan(humidity))
    {
        Serial.println("DHT11 SENSOR ERROR!");
        digitalWrite(BUZZER_PIN, BUZZER_ON);
        buzzerStatusFeed->save("ON");
        refrigeratorStatusFeed->save("ALERT");
        delay(2000);
        return;
    }
    Serial.println();
    Serial.println();
    Serial.print("Temperature : ");

```

```

Serial.print(temperature);
Serial.println(" °C");
Serial.print("Humidity   : ");
Serial.print(humidity);
Serial.println(" %");
bool temperatureAlert = false;
bool humidityAlert = false;
if (temperature > TEMP_MAX || temperature < TEMP_MIN)
{
    temperatureAlert = true;
}
if (humidity > HUM_MAX || humidity < HUM_MIN)
{
    humidityAlert = true;
}
if (temperatureAlert || humidityAlert)
{
    digitalWrite(BUZZER_PIN, BUZZER_ON);
    refrigeratorStatusFeed->save("ALERT");
    buzzerStatusFeed->save("ON");
    Serial.println("STATUS      : ALERT");
    Serial.println("BUZZER      : ON");
}
else
{
    digitalWrite(BUZZER_PIN, BUZZER_OFF);
    refrigeratorStatusFeed->save("NORMAL");
    buzzerStatusFeed->save("OFF");
    Serial.println("STATUS      : NORMAL");
    Serial.println("BUZZER      : OFF");
}
temperatureFeed->save(temperature);
humidityFeed->save(humidity);
Serial.println("Data sent to Adafruit IO");
Serial.println();
delay(5000);
}

```


9. System Working:

When the system is powered ON, the ESP32 initializes the DHT11 sensor and establishes a Wi-Fi connection.

The DHT11 sensor continuously senses the environmental conditions inside the refrigerator. It provides two main measurements:

- Temperature
- Relative humidity

The ESP32 receives these measurements and processes them.

For example,

the system may display:

Temperature: 4.5 °C

Humidity: 55 % RH

Status: Normal

The readings can be displayed locally on the OLED screen. At the same time, the ESP32 sends the data to the IoT cloud through Wi-Fi.

The user can view the data remotely through a dashboard.

If the temperature increases significantly, the system identifies the condition as abnormal and activates the warning system.

For example:

Temperature: 10 °C

Humidity: 70 % RH

Status: Warning

This warning can indicate that the refrigerator requires attention.

Possible reasons for an abnormal temperature include:

- Refrigerator door left open.
- Frequent door opening.
- Excessive food loading.
- Poor air circulation.
- Power interruption.
- Insufficient cooling.
- Refrigerator malfunction.

The system only monitors the environmental conditions. The exact mechanical fault must be identified separately.

```

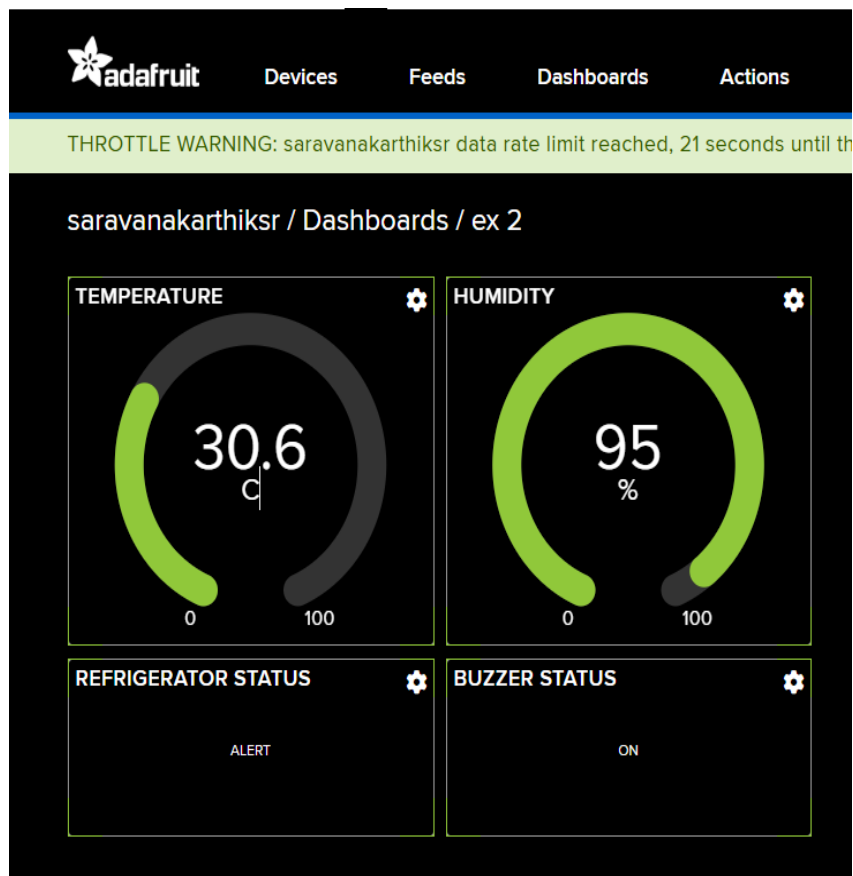
Adafruit IO Connected!
-----
Temperature Limit: 2.00 - 8.00 °C
Humidity Limit: 30.00 - 70.00 %
-----

===== REFRIGERATOR DATA =====
Temperature : 28.50 °C
Humidity    : 70.20 %
HVAC STATUS : ALERT
BUZZER      : ON
Data sent to Adafruit IO
=====

===== REFRIGERATOR DATA =====
Temperature : 28.50 °C
Humidity    : 70.60 %
HVAC STATUS : ALERT
BUZZER      : ON
Data sent to Adafruit IO
=====

===== REFRIGERATOR DATA =====
Temperature : 28.50 °C
Humidity    : 70.60 %
HVAC STATUS : ALERT
BUZZER      : ON
Data sent to Adafruit IO
=====

```



10. Bill of Materials:

COMPONENT	QUANTITY
ESP8266 NodeMCU	1
DHT22 Sensor	1
Relay Module	1
OLED/LCD Display	1
Buzzer	1
Breadboard	1
Jumper Wires	Required

11. Prototype Implementation:

The prototype is implemented step-by-step to ensure proper operation of the smart refrigerator monitoring system.

Stage 1 – Hardware Assembly

ESP32, DHT11 sensor, OLED display, LED, and buzzer are connected according to the circuit design.

Power is supplied and each component is checked for proper operation.

Stage 2 – Sensor Calibration

The DHT11 sensor is placed in the refrigerator to measure temperature and humidity.

The readings are observed through the ESP32 Serial Monitor and checked for stability.

Stage 3 – OLED Monitoring

The OLED display is programmed to show real-time temperature and humidity values.

The refrigerator condition is displayed as **NORMAL** or **WARNING**.

Stage 4 – Condition Monitoring

Preset temperature and humidity limits are programmed into the ESP32.

The controller continuously compares the sensor readings with these limits.

Stage 5 – Alert Operation

When the measured values exceed the safe limits, the ESP32 activates the LED and buzzer.

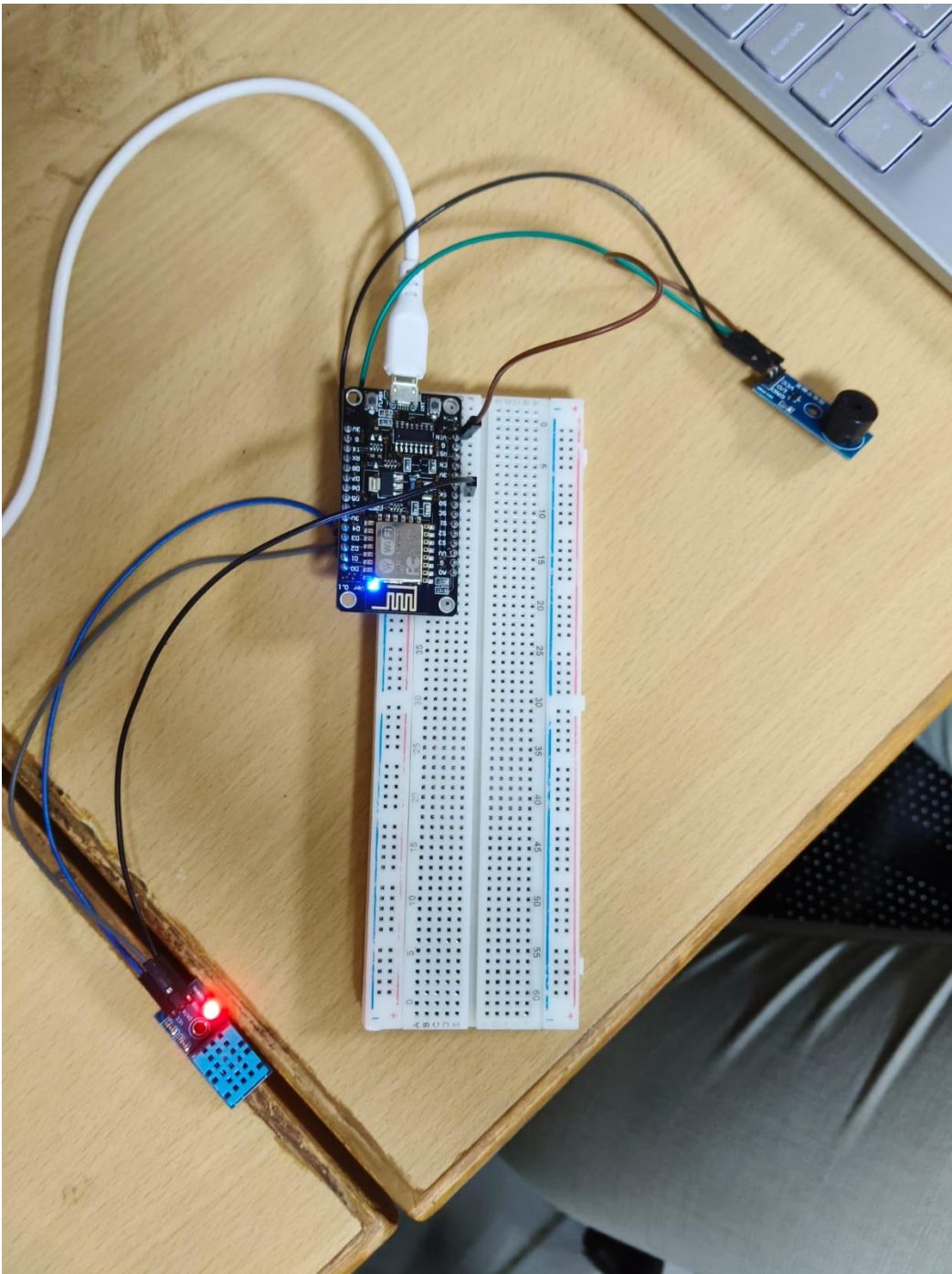
This immediately alerts the user about abnormal refrigerator conditions.

Stage 6 – IoT Data Transmission

The ESP32 connects to Wi-Fi and sends temperature and humidity data to the IoT platform. The user can monitor the refrigerator remotely using a dashboard.

Stage 7 – Final Testing

The complete system is tested under different refrigerator conditions. Sensor readings, OLED display, alerts, Wi-Fi communication, and dashboard operation are verified.



12.Results & Performance Analysis:

The developed **Smart Refrigerator Monitoring System** successfully monitors the internal temperature and humidity of the refrigerator using the DHT11 sensor. The ESP32 collects the sensor readings and processes the information continuously.

The system provides real-time information about the refrigerator environment. The temperature and humidity values can be displayed on the OLED screen and transmitted to an IoT dashboard through Wi-Fi. This allows the user to monitor the refrigerator remotely.

During normal operation, the system displays the current environmental conditions and indicates that the refrigerator is operating within the selected range. If the temperature or humidity exceeds the predefined limit, the warning system is activated.

For example, when the refrigerator temperature remains within the required range, the system shows a normal status. If the temperature increases due to frequent door opening or insufficient cooling, the system detects the change and provides a warning.

Continuous monitoring also helps identify temperature variations over time. Historical data can be analyzed to determine how the refrigerator performs during different operating conditions. This information can help identify abnormal operation and support better energy-management decisions.

The system can also help reduce food spoilage by providing an early warning when the internal temperature becomes unsuitable for food storage. Instead of depending only on manual inspection, the user receives continuous information about the refrigerator conditions.

The proposed system is simple and economical because it uses easily available components such as the DHT11 sensor and ESP32. The built-in Wi-Fi capability of the ESP32 eliminates the need for a separate Wi-Fi module.